

Docker Volumes Hands-On Lab (HOL)

Trainer: Dr. Sandeep Kumar Sharma

Learning Objectives

- Understand what Docker Volumes are
 - Understand why persistent storage is required in containers
 - Learn how Docker volumes store data outside containers
 - Learn how multiple containers can share the same data
 - Understand Bind Mounts vs Docker Volumes
 - Learn how to mount both a Docker volume and a bind mount in the same container
-

Learning Outcomes

After completing this lab, participants will be able to:

- Explain Docker persistent storage
 - Create and manage Docker volumes
 - Mount volumes inside containers
 - Share storage between containers
 - Verify data persistence after container deletion
 - Implement bind mounts with host directories
 - Use both volume and bind mount in a single container
-

What is Docker Volume?

By default, containers are **ephemeral**.

This means when a container is deleted, all the data inside the container is also deleted.

This becomes a problem when applications need **persistent storage**, for example:

- Databases
- Logs
- Application uploads
- Shared configuration files

Docker solves this problem using **Volumes**.

A Docker Volume is a **persistent storage location managed by Docker** which exists outside the container filesystem.

Even if the container is deleted, the data stored inside the volume **remains safe**.

Docker stores volumes in the host system at:

```
/var/lib/docker/volumes/
```

Why Do We Use Docker Volumes?

1. Persistent data storage
2. Data sharing between containers
3. Backup and migration of data
4. Decoupling data from container lifecycle
5. Better performance than container filesystem

LAB PART 1 – List Existing Docker Volumes

```
docker volume ls
```

This command shows all volumes available in the Docker host.

LAB PART 2 – Create a Docker Volume

```
docker volume create training_volume
```

Explanation:

- **docker volume create** → creates a Docker-managed storage
- **training_volume** → name of the volume

Verify:

```
docker volume ls
```

LAB PART 3 – Verify Volume Location on Host

Navigate to Docker volume storage path.

```
cd /var/lib/docker/volumes/  
ls
```

You will see the volume **training_volume**.

Explore the directory:

```
cd training_volume  
cd _data  
ls
```

This is the actual directory where Docker stores the volume data.

LAB PART 4 – Create File Inside Volume

```
touch devops_notes.txt  
  
echo "Docker volume persistent data created during training" > devops_notes.txt
```

Verify:

```
ls  
cat devops_notes.txt
```

LAB PART 5 – Mount Volume Inside Container

Run first container:

```
docker run -it --name volume_container1 -v training_volume:/app_storage ubuntu
```

Explanation:

- **-v** → volume flag
- **training_volume** → Docker volume
- **/app_storage** → mount point inside container

Inside container:

```
cd /app_storage  
ls
```

You should see:

```
devops_notes.txt
```

Exit the container.

LAB PART 6 – Share Same Volume With Another Container

Run second container:

```
docker run -it --name volume_container2 -v training_volume:/data ubuntu
```

Inside container:

```
cd /data  
ls
```

You will again see:

```
devops_notes.txt
```

This proves:

Multiple containers can share the same Docker volume.

LAB PART 7 – Remove Containers

```
docker rm -f $(docker ps -aq)
```

LAB PART 8 – Verify Data Persistence

Check the volume directory again:

```
cd /var/lib/docker/volumes/  
cd training_volume/_data  
ls
```

The file **devops_notes.txt** will still exist.

This proves that **Docker volumes persist even after containers are deleted.**

LAB PART 9 – Bind Mount Example

Create a directory on the host machine.

```
mkdir host_project_data  
cd host_project_data
```

Create some files.

```
touch config.txt app.log  
mkdir service1 service2  
ls
```

LAB PART 10 – Run Container Using Bind Mount

```
docker run -it --name bind_container --mount type=bind,source="/root/  
host_project_data",target="/projectdata" ubuntu
```

Inside container:

```
cd /projectdata  
ls
```

You will see the same host files inside the container.

LAB PART 11 – Bind Mount With NGINX Container

```
docker run -d --name web_container --mount type=bind,source="/root/  
host_project_data",target="/usr/share/nginx/html" nginx
```

Verify running container:

```
docker ps
```

Access container shell:

```
docker exec -it web_container /bin/bash
```

LAB PART 12 – Mount Volume and Bind Mount Together

Sometimes applications require:

- **Volume for application data**
- **Bind mount for configuration files**

Docker allows mounting both in the same container.

Example:

```
docker run -it  
--name hybrid_container  
-v training_volume:/appdata
```

```
--mount type=bind,source="/root/host_project_data",target="/configdata"
ubuntu
```

Inside container:

```
cd /appdata
ls

cd /configdata
ls
```

Now the container is using:

- **Docker Volume** → **/appdata**
- **Bind Mount** → **/configdata**

This is very common in real production applications.

Example use case:

- Volume → database storage
- Bind mount → configuration files

LAB Summary

In this lab we learned:

- Docker persistent storage
- Docker volume creation and management
- Mounting volumes inside containers
- Sharing volumes across containers
- Data persistence after container deletion
- Bind mounts with host directories
- Using both **volume + bind mount** in a single container

Key Concept

Containers are temporary.

Volumes are permanent.

Always store **important application data inside volumes, not inside containers.**